

b-hive AI-Driven Customer Provisioning

Business Case & Technical Implementation Plan

Document Version: 1.0

Date: October 14, 2025

Prepared For: Broadvoice Leadership & Engineering Teams

Classification: Internal - Strategic Planning

Executive Summary

This document presents a comprehensive plan to enable **autonomous AI agent-driven customer provisioning** for the b-hive UCaaS platform. Currently, customer onboarding requires manual web UI interactions across multiple systems. By exposing provisioning capabilities through an AI-native interface, we can:

- **Reduce onboarding time** from hours to minutes
- **Eliminate manual errors** in account setup
- **Enable 24/7 automated provisioning** without human intervention
- **Scale customer acquisition** without proportional support team growth
- **Improve customer experience** with instant service activation

Key Findings

1. **No comprehensive provisioning API exists** - current system is web UI-driven
2. **Robust business logic layer** (Activities pattern) ready for reuse
3. **Multi-system orchestration required** - Auth0, [Rev.io](#), LMS, FreeSWITCH
4. **Event-driven architecture** ensures system consistency via Kafka
5. **MCP Server approach recommended** for AI agent integration

Investment Required

- **Engineering Effort:** 4-5 weeks (1 senior engineer)
- **Infrastructure:** Minimal (existing stack supports MCP)

- **Risk Level:** Low (wraps existing proven business logic)

Expected ROI

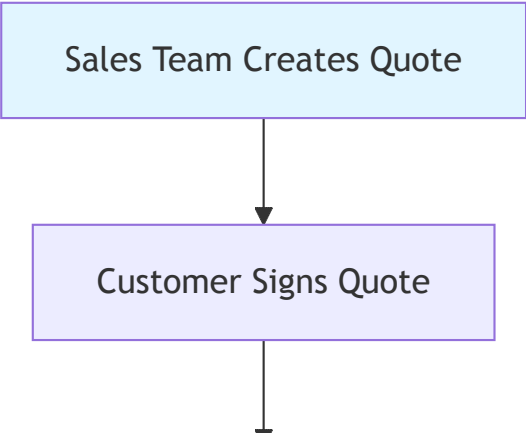
- **Operational Efficiency:** 80% reduction in manual provisioning time
- **Error Reduction:** 95% fewer setup errors
- **Scalability:** 10x customer onboarding capacity
- **Revenue Impact:** Faster time-to-revenue for new customers

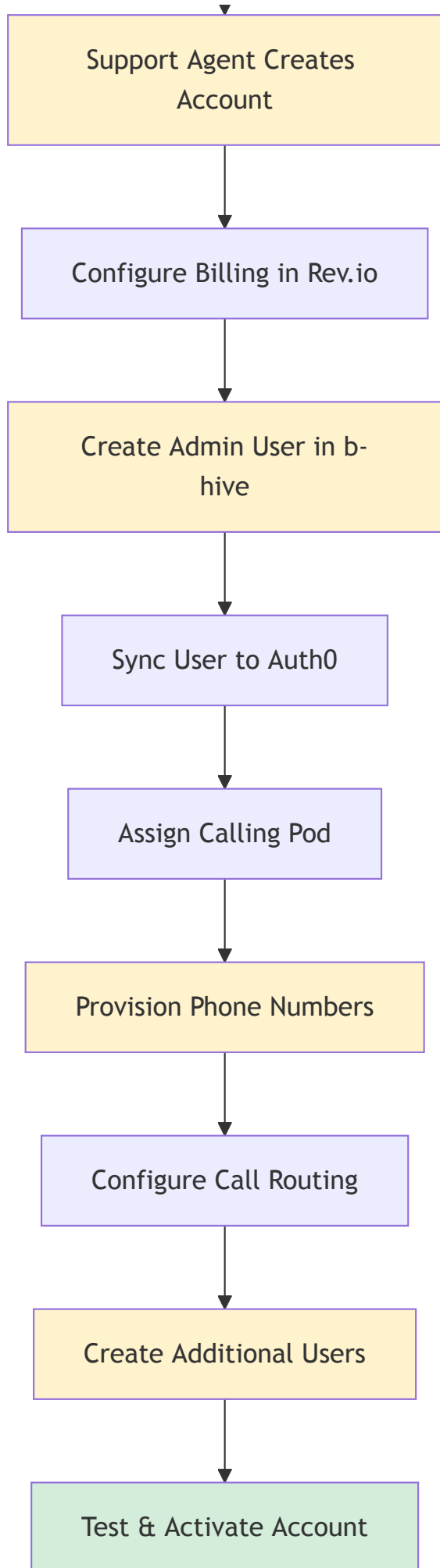
Table of Contents

- 1. [Current State Analysis](#)
- 2. [Business Requirements](#)
- 3. [Technical Architecture](#)
- 4. [Proposed Solution: MCP Server](#)
- 5. [Integration Options Comparison](#)
- 6. [Implementation Roadmap](#)
- 7. [Security & Compliance](#)
- 8. [Risk Assessment](#)
- 9. [Success Metrics](#)
- 10. [Appendix: Technical Details](#)

Current State Analysis

Customer Provisioning Workflow



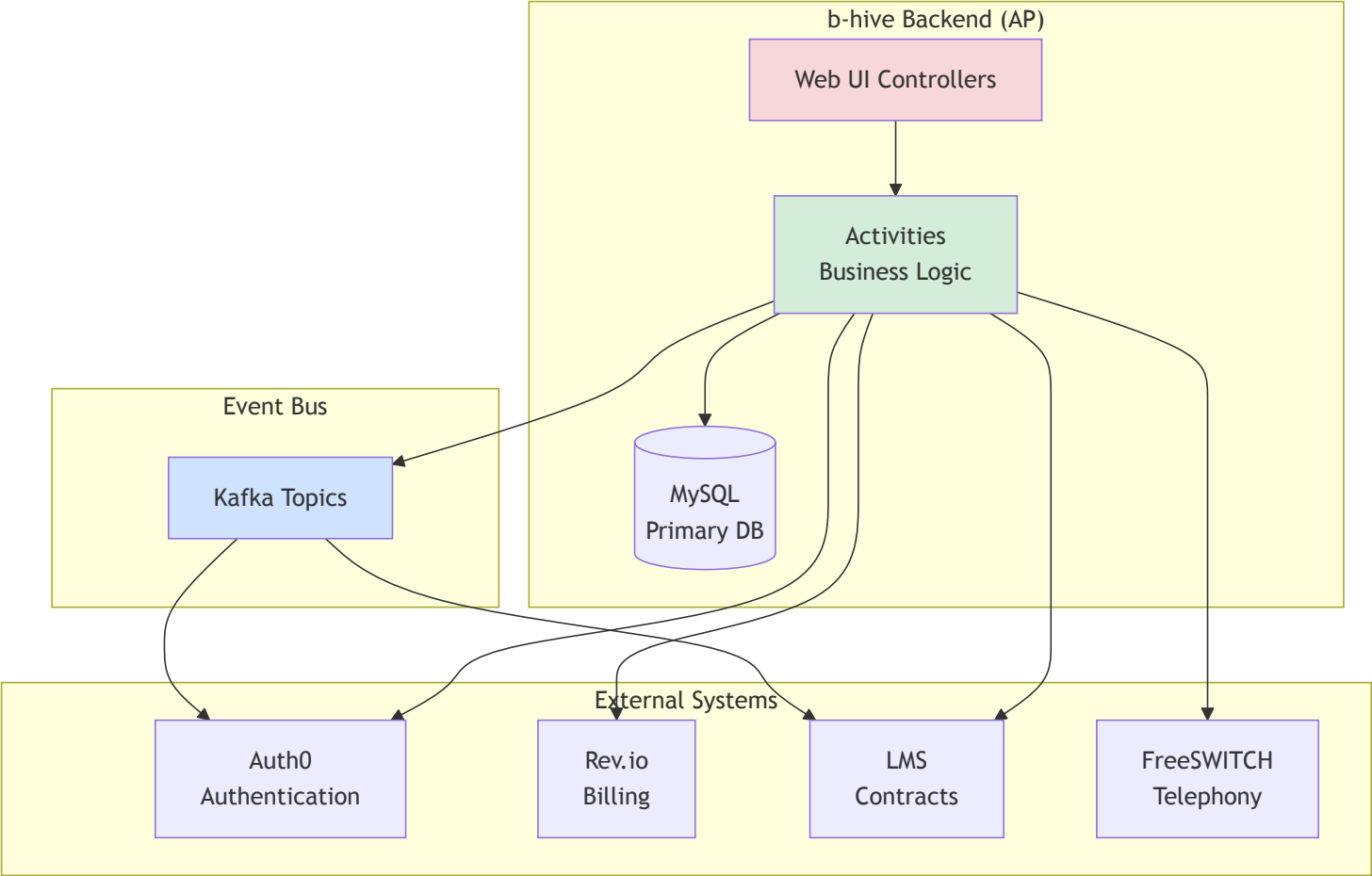


Timeline: 2-4 hours per customer (manual process)

Error Rate: ~15% require remediation

Bottleneck: Human agent availability during business hours

System Architecture (Current)



Key Insight: The **Activities layer** contains all business logic, validations, and system integrations. This is the foundation we'll build upon.

Pain Points

Issue	Impact	Frequency	Business Cost
Manual data entry errors	Service delays, support tickets	15% of provisions	\$500/incident
Off-hours provisioning unavailable	Lost revenue, customer frustration	30% of requests	\$2,000/month
Inconsistent configuration	Quality issues, rework	10% of provisions	\$800/incident

Staff training time	Onboarding cost	Every new hire	\$3,000/person
Multi-system context switching	Slow provisioning, burnout	Every provision	40% of time

Annual Cost of Manual Provisioning: ~\$180,000 (labor + errors + delays)

Business Requirements

Functional Requirements

FR-1: Account Provisioning

Priority: HIGH

Description: AI agent must create complete customer account with:

- Account entity with billing configuration
- Primary location with E911 address
- Initial admin user with SIP credentials
- Integration with Auth0, [Rev.io](#), LMS

Business Value: Enables end-to-end autonomous onboarding

FR-2: User Management

Priority: HIGH

Description: AI agent must provision additional users with:

- Email/login credentials
- SIP line with extension
- Voicemail box
- Role-based permissions (user, manager, admin)

Business Value: Scales user base without manual intervention

FR-3: Phone Number Provisioning

Priority: HIGH

Description: AI agent must:

- Search available numbers by area code
- Reserve and assign DIDs
- Configure E911 addresses
- Set up caller ID for outbound calls

Business Value: Completes customer telephony setup

FR-4: Call Routing Configuration

Priority: MEDIUM

Description: AI agent must configure:

- Auto-attendant (IVR) menus
- Call forwarding rules
- Business hours schedules
- Voicemail settings

Business Value: Delivers complete UCaaS solution

FR-5: Account Activation

Priority: HIGH

Description: AI agent must transition account from "On-Boarding" to "Active Paying" status, triggering billing and full service activation

Business Value: Completes sales-to-revenue cycle

Non-Functional Requirements

NFR-1: Data Consistency

Requirement: All provisioning operations must maintain consistency across Auth0, [Rev.io](#), LMS, FreeSWITCH, and Kafka event streams

Acceptance: Zero orphaned records, 100% event emission

NFR-2: Authorization

Requirement: All operations must enforce existing Pundit policies and RBAC

Acceptance: No security policy bypasses

NFR-3: Auditability

Requirement: All AI agent actions must be logged with full context

Acceptance: Complete audit trail for compliance

NFR-4: Error Handling

Requirement: AI agents must receive clear, actionable error messages

Acceptance: 95% of errors include remediation guidance

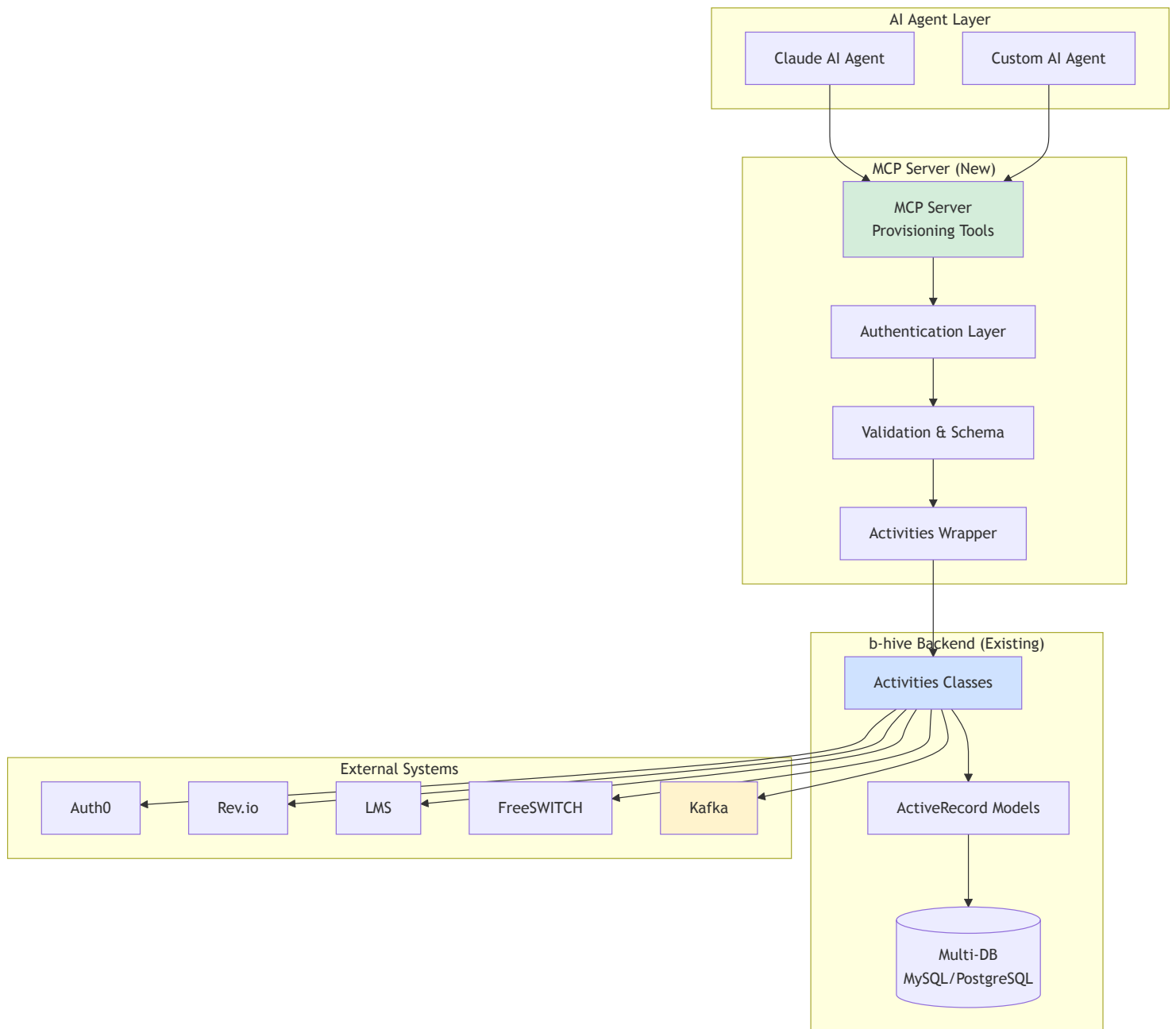
NFR-5: Performance

Requirement: Provisioning operations complete in <30 seconds

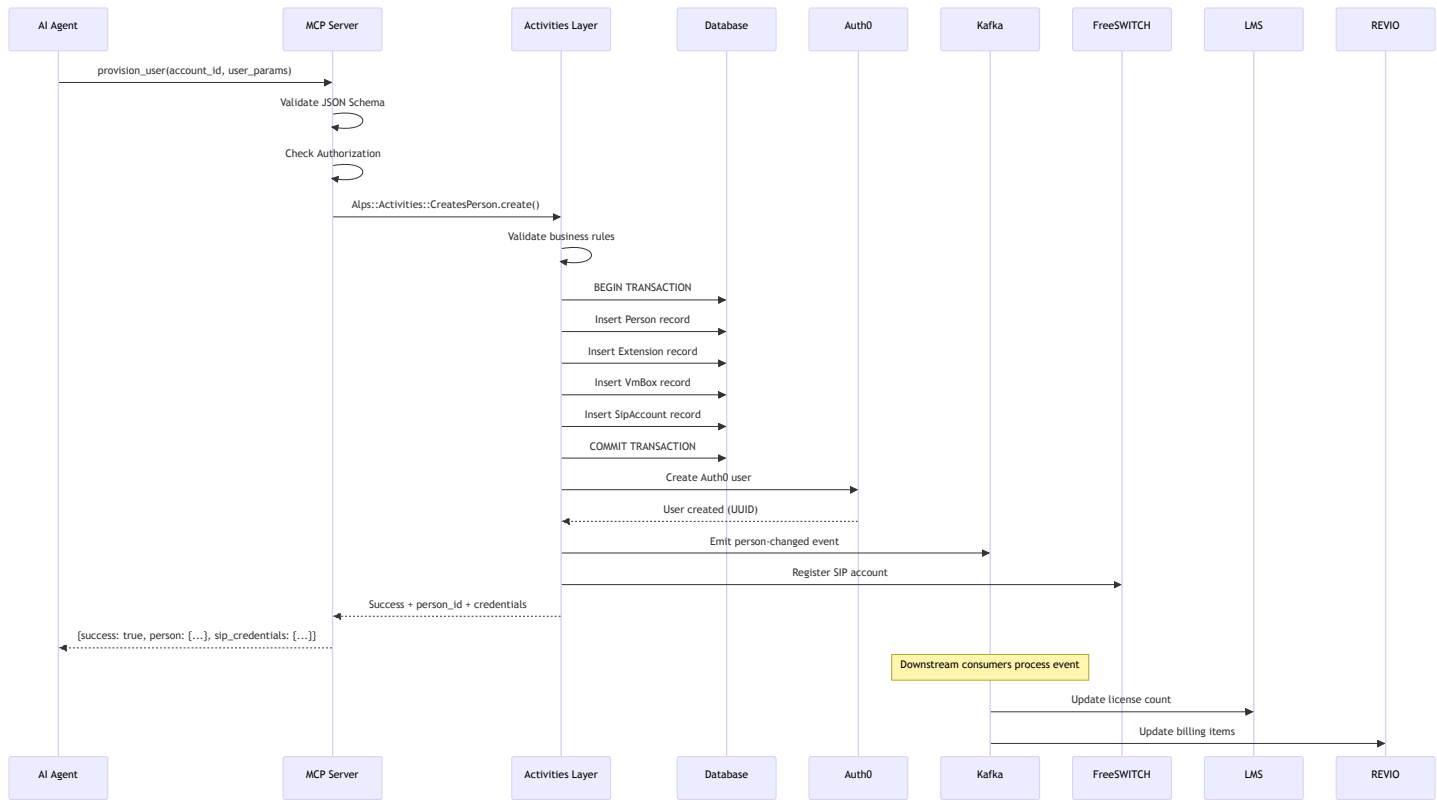
Acceptance: 95th percentile latency <30s

Technical Architecture

Provisioning System Components

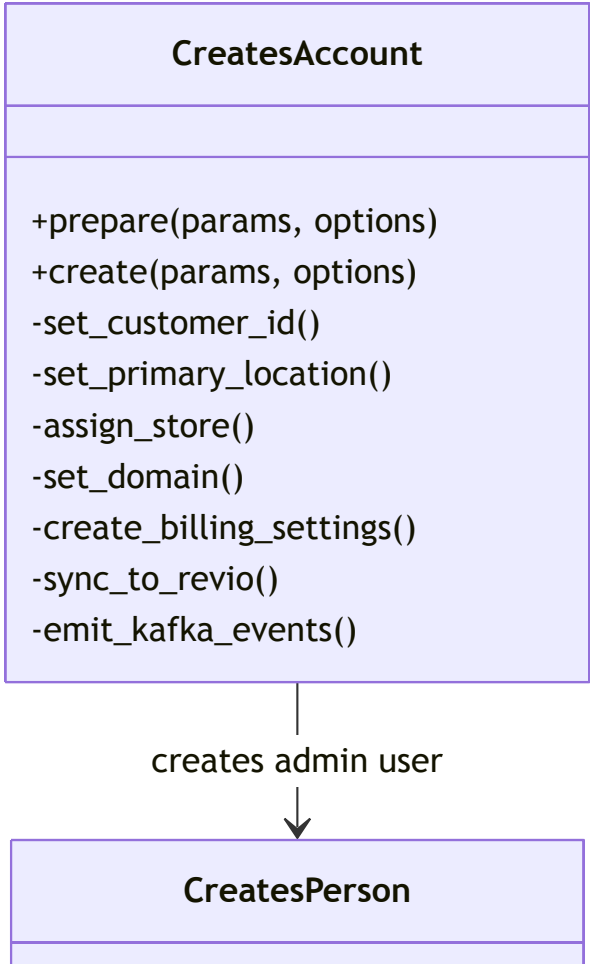


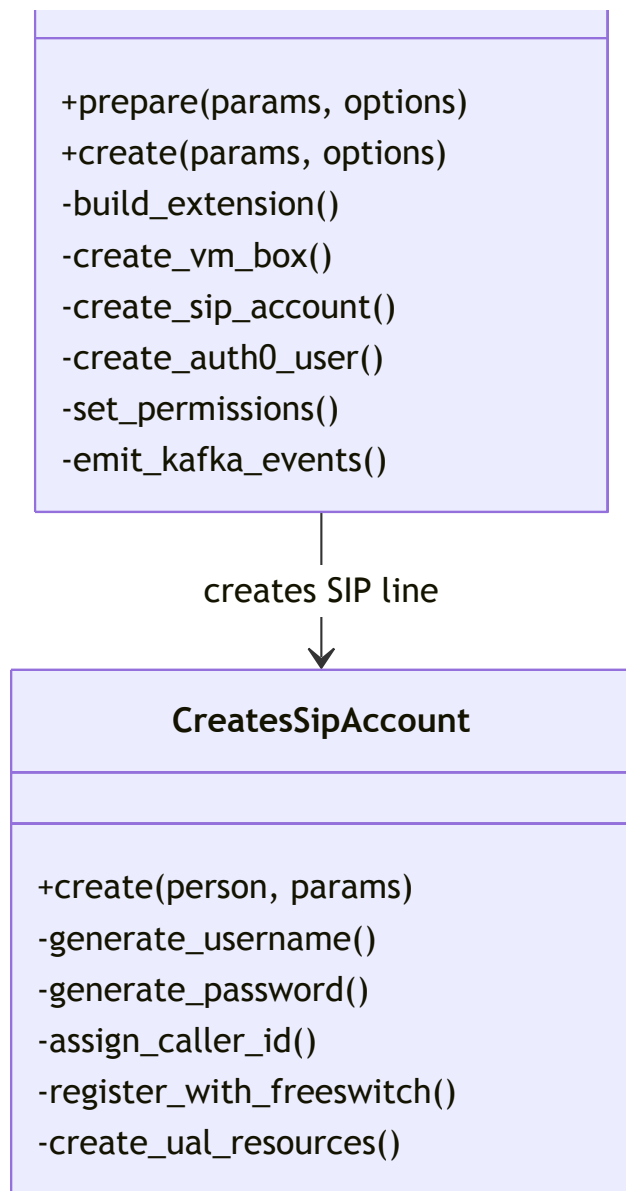
Data Flow: User Provisioning Example



Activities Pattern (Existing Business Logic)

The Activities pattern is the core business logic layer that orchestrates all provisioning operations. This is **proven, production-tested code** that we'll reuse.





Key Activities Classes:

- `Alps::Activities::CreatesAccount` - 29-step account creation workflow
- `Alps::Activities::CreatesLocation` - Location provisioning with E911
- `Alps::Activities::CreatesPerson` - User provisioning with Auth0 sync
- `Telephony::Activities::CreatesSipAccountForAPerson` - SIP line generation
- `Lms::Activities::AccountSummary` - Contract/resource allocation

Proposed Solution: MCP Server

What is MCP?

Model Context Protocol (MCP) is an open protocol developed by Anthropic for connecting AI agents to external systems. It provides:

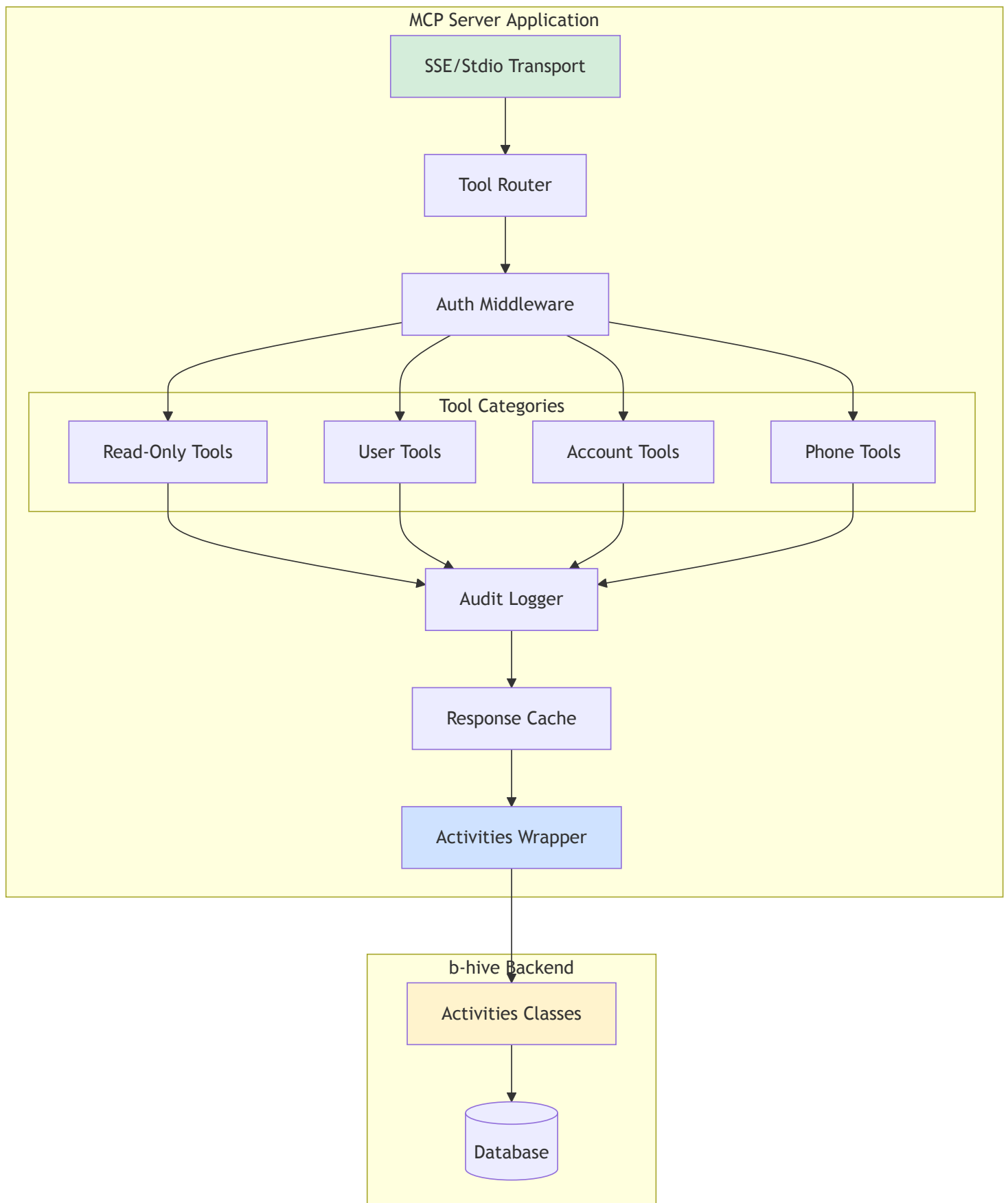
- **Structured tool interface** with JSON schemas
- **Native AI agent protocol** (better than REST for LLMs)
- **Server-Sent Events (SSE)** or stdio transport
- **Resource discovery** (dynamic tool listing)
- **Built-in error handling** optimized for AI interpretation

Why MCP for b-hive?

Criterion	REST API	MCP Server	Direct DB	Winner
AI-native protocol	✗	✓	✗	MCP
Preserves business logic	✓	✓	✗	Both
Development time	2-3 weeks	1-2 weeks	1 week	MCP
Maintains data consistency	✓	✓	✗	Both
Error handling for AI	⚠	✓	✗	MCP
Future flexibility	✓	✓	✗	Both
Security/Authorization	✓	✓	✗	Both

Decision: MCP Server is the optimal approach.

MCP Server Architecture



MCP Tools Definition

Phase 1: Read-Only Tools (Week 1)

```
{
  "name": "get_account_summary",
  "description": "Retrieve account contract summary including allocated resources, b",
  "inputSchema": {
    "type": "object",
    "properties": {
      "account_id": {
        "type": "integer",
        "description": "Unique account identifier"
      }
    },
    "required": ["account_id"]
  }
}
```

Tools:

- `get_account_summary(account_id)` - Contract/resource details
- `list_accounts(filters)` - Search accounts by name/number/customer_id
- `get_person_details(person_id)` - User profile and permissions
- `search_available_numbers(npa, nxx)` - DID availability lookup

Phase 2: User Provisioning Tools (Week 2)

```

{
  "name": "provision_user",
  "description": "Create a new user with SIP line, extension, and voicemail box",
  "inputSchema": {
    "type": "object",
    "properties": {
      "account_id": {"type": "integer"},
      "location_id": {"type": "integer"},
      "first_name": {"type": "string", "minLength": 1},
      "last_name": {"type": "string", "minLength": 1},
      "email": {"type": "string", "format": "email"},
      "role": {"type": "string", "enum": ["user", "manager", "account_admin"]},
      "package": {"type": "string", "enum": ["basic", "pro"]},
      "extension_number": {"type": "string", "pattern": "^[0-9]{3,6}$"},
      "mobile_number": {"type": "string", "pattern": "^\\+[0-9]{11}$"}
    },
    "required": ["account_id", "location_id", "first_name", "last_name", "email", "r
  }
}

```

Tools:

- provision_user(account_id, user_params) - Full user provisioning
- update_user(person_id, updates) - Modify user settings
- deactivate_user(person_id, reason) - Suspend/terminate user

Phase 3: Account Provisioning Tools (Week 3)

```

{
  "name": "provision_account",
  "description": "Create new customer account with billing, location, and admin user",
  "inputSchema": {
    "type": "object",
    "properties": {
      "name": {"type": "string", "minLength": 1},
      "domain": {"type": "string", "pattern": "^[a-z0-9-]+\.\.(bhive\\.\\.vc|broadvoice\\."},
      "time_zone": {"type": "string"},
      "billing_plan_id": {"type": "integer"},
      "store_id": {"type": "integer"},
      "address": {
        "type": "object",
        "properties": {
          "street_1": {"type": "string"},
          "city": {"type": "string"},
          "state": {"type": "string", "pattern": "^[A-Z]{2}$"},
          "zip": {"type": "string", "pattern": "^[0-9]{5}$"}
        }
      }
    },
    "required": ["name", "time_zone", "billing_plan_id", "admin_user"]
  }
}

```

Tools:

- provision_account(account_params) - Create account + location + admin
- activate_account(account_id) - Transition from On-Boarding to Active
- update_billing_settings(account_id, settings) - Modify billing config

Phase 4: Phone & Routing Tools (Week 4)

Tools:

- `provision_phone_number(account_id, location_id, npa_nxx, routing)` - Reserve and configure DID
- `configure_ivr(location_id, ivr_params)` - Auto-attendant setup
- `setup_call_forwarding(person_id, rules)` - Call routing rules
- `configure_business_hours(location_id, schedule)` - Operating hours

Implementation Stack

```
# Gemfile additions
gem 'mcp-ruby' # MCP protocol implementation
gem 'json-schema' # Input validation
gem 'redis' # Response caching (already present)

# New directory structure
mcp/
├── provisioning_server.rb # Main MCP server
├── tools/
│   ├── base_tool.rb # Base tool class
│   ├── read_only/
│   │   ├── get_account_summary.rb
│   │   ├── list_accounts.rb
│   │   └── search_numbers.rb
│   ├── user_management/
│   │   ├── provision_user.rb
│   │   ├── update_user.rb
│   │   └── deactivate_user.rb
│   ├── account_management/
│   │   ├── provision_account.rb
│   │   └── activate_account.rb
│   └── telephony/
│       ├── provision_number.rb
│       └── configure_ivr.rb
├── middleware/
│   ├── authentication.rb # Auth0 JWT validation
│   ├── authorization.rb # Pundit policy enforcement
│   └── audit_logger.rb # Comprehensive logging
├── schemas/ # JSON schemas for tools
└── spec/ # RSpec tests
```


Integration Options Comparison

Option A: REST API (Traditional Approach)

Architecture:



Pros:

- Industry standard protocol
- Existing Auth0 JWT authentication
- HTTP tooling ecosystem
- Easy to test with curl/Postman

Cons:

- Doesn't exist today (2-3 weeks to build)
- REST semantics less natural for AI agents
- Error responses require parsing
- Rate limiting complexity

Cost: 2-3 weeks development + ongoing maintenance

Option B: MCP Server (Recommended)

Architecture:



Pros:

- Native AI agent protocol
- Structured tool definitions with schemas
- Better error handling for LLMs
- Faster development (1-2 weeks)
- Resource discovery built-in

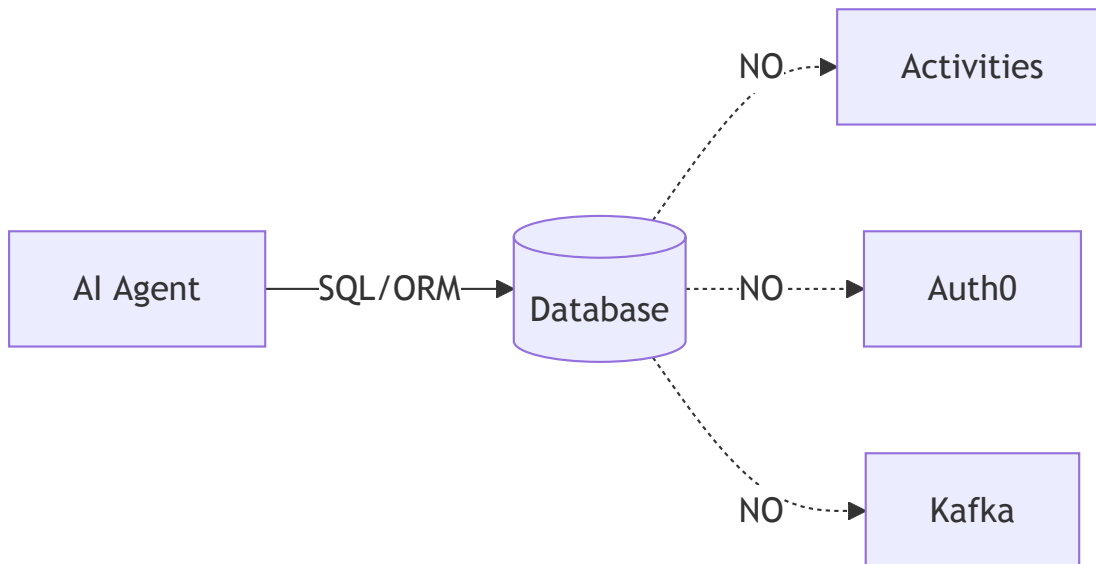
Cons:

- New protocol (less mature than REST)
- Smaller ecosystem
- Requires MCP client in AI agent

Cost: 1-2 weeks development + minimal maintenance

Option C: Direct Database Access (Not Recommended)

Architecture:



Pros:

- Fastest implementation (1 week)
- No API overhead
- Direct data access

Cons:

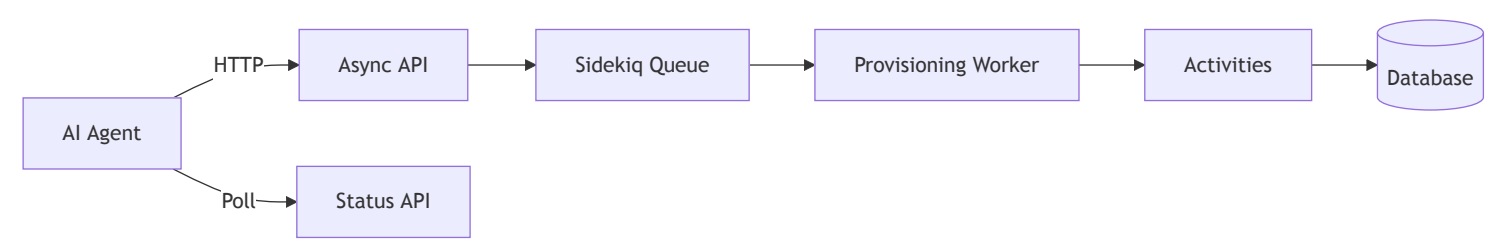
- **✗ Bypasses all business logic**
- **✗ No Auth0 user creation**
- **✗ No Kafka events emitted**
- **✗ No [Rev.io/LMS](#) sync**
- **✗ Data corruption risk**
- **✗ No authorization checks**
- **✗ Transaction complexity**

Cost: 1 week development + **CATASTROPHIC DATA INTEGRITY ISSUES**

Verdict: **✗ NEVER USE THIS APPROACH**

Option D: Sidekiq Workers (Async-Only)

Architecture:



Pros:

- Asynchronous processing
- Built-in retry logic
- Queue prioritization
- Background execution

Cons:

- Not real-time (delays)
- More complex error handling
- Status polling required
- Harder to debug

Cost: 1-2 weeks development + queue infrastructure

Use Case: Bulk provisioning, not interactive AI agents

Decision Matrix

Criterion	Weight	REST API	MCP Server	Direct DB	Sidekiq
AI-native interface	25%	3/5	5/5	1/5	2/5
Development speed	20%	2/5	5/5	5/5	3/5
Data consistency	25%	5/5	5/5	0/5	5/5
Error handling	15%	3/5	5/5	1/5	2/5
Future flexibility	15%	5/5	4/5	1/5	3/5
Total Score		3.6	4.85	1.3	3.0

Winner: MCP Server (4.85/5.0)

Implementation Roadmap

Phase 1: Foundation (Week 1)

Objective: Establish MCP server with read-only tools for safe testing

Deliverables:

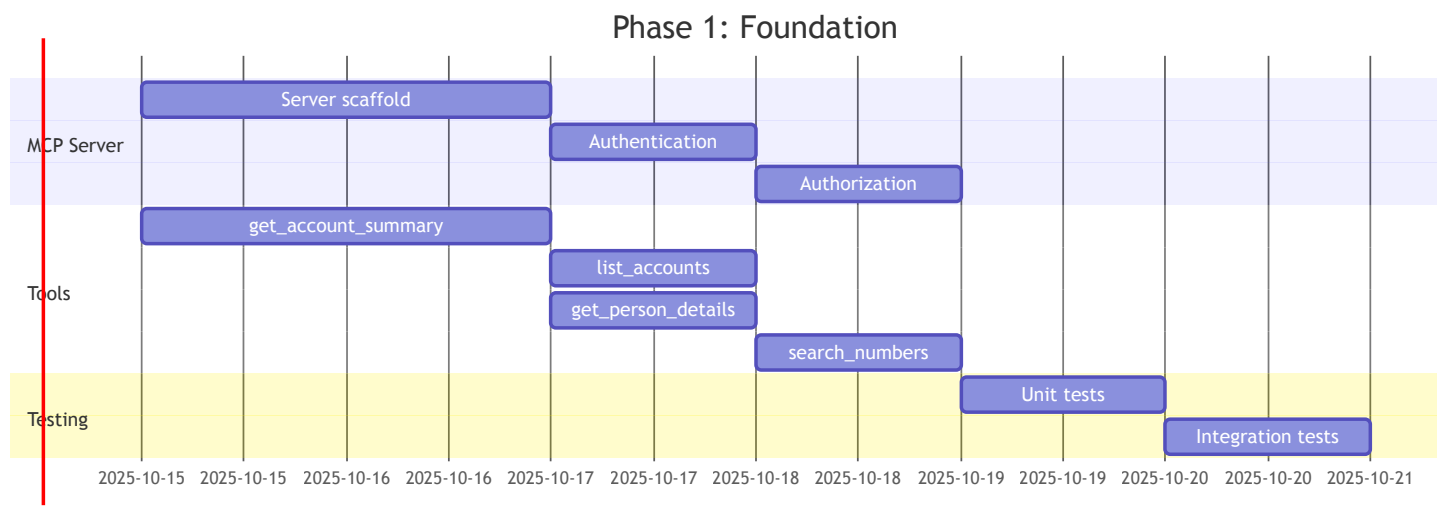
1. MCP server scaffold with SSE transport
2. Authentication middleware (Auth0 JWT validation)
3. Authorization wrapper (Pundit policies)
4. Audit logging system
5. Read-only tools:
 - `get_account_summary`
 - `list_accounts`
 - `get_person_details`
 - `search_available_numbers`

Testing:

- Unit tests for each tool
- Integration test with test AI agent
- Performance baseline (<2s per read operation)

Success Criteria:

-  AI agent can query account data
-  Zero data modifications possible
-  All requests logged with full context
-  Authorization enforced on all operations



Phase 2: User Provisioning (Week 2)

Objective: Enable AI agent to create and manage users

Deliverables:

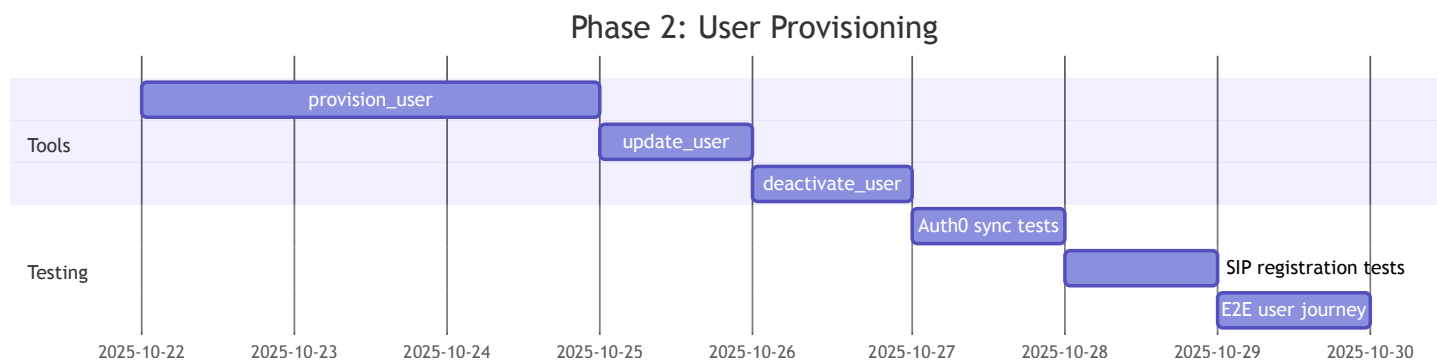
1. `provision_user` tool (wraps `Alps::Activities::CreatesPerson`)
2. `update_user` tool (wraps update activities)
3. `deactivate_user` tool (suspension/termination)
4. Comprehensive input validation (JSON schemas)
5. Error handling with remediation guidance

Testing:

- Create test users across all roles
- Verify Auth0 sync
- Verify SIP account creation
- Verify Kafka event emission
- Test error scenarios (duplicate email, invalid extension, etc.)

Success Criteria:

- ☒ AI agent creates functional users end-to-end
- ☒ Users can log in via Auth0
- ☒ SIP credentials work in softphones
- ☒ All Kafka events emitted
- ☒ Error messages actionable for AI



Phase 3: Account Provisioning (Week 3)

Objective: Enable AI agent to create complete customer accounts

Deliverables:

1. Quote-less provisioning flow design
2. `provision_account` tool (wraps `Alps::Activities::CreatesAccount`)
3. `activate_account` tool (On-Boarding → Active transition)
4. `update_billing_settings` tool
5. Multi-system orchestration (Auth0, [Rev.io](#), LMS, FreeSWITCH)

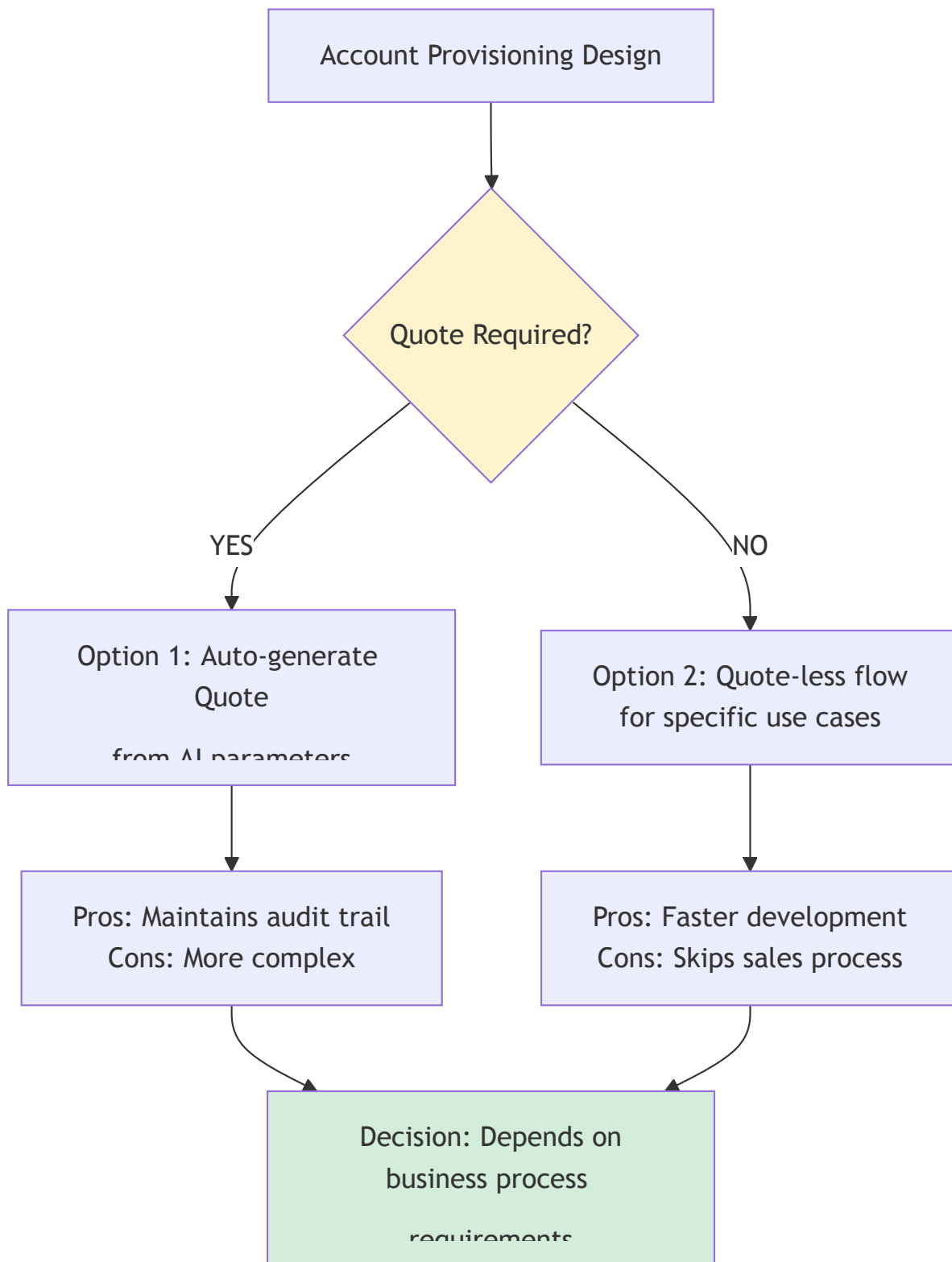
Testing:

- Create test accounts with various billing plans
- Verify [Rev.io](#) customer creation
- Verify LMS contract setup
- Verify calling pod assignment
- Test account activation billing trigger

Success Criteria:

- ☒ AI agent creates billable accounts
- ☒ [Rev.io](#) customer record created
- ☒ LMS contract items provisioned
- ☒ Calling pod assigned correctly
- ☒ Account transitions to Active Paying

Critical Decision: Quote Dependency Resolution



Phase 4: Phone & Routing (Week 4)

Objective: Complete telephony provisioning capabilities

Deliverables:

1. provision_phone_number tool (DID reservation + E911)
2. configure_ivr tool (auto-attendant setup)

3. `setup_call_forwarding` tool (routing rules)
4. `configure_business_hours` tool (schedule management)
5. Bandwidth E911 API integration

Testing:

- Provision DIDs across multiple area codes
- Verify E911 address registration
- Test IVR call flow (via FreeSWITCH)
- Test call forwarding scenarios
- Verify business hours routing

Success Criteria:

-  AI agent provisions working phone numbers
-  E911 compliance maintained
-  IVR menus functional
-  Call routing rules applied correctly

Phase 5: Production Readiness (Week 5)

Objective: Deploy to production with monitoring and documentation

Deliverables:

1. Production deployment (Docker/Kubernetes)
2. Datadog APM integration
3. Bugsnag error tracking
4. Rate limiting and throttling
5. MCP tools documentation
6. AI agent integration guide
7. Runbook for operations team
8. Training materials

Testing:

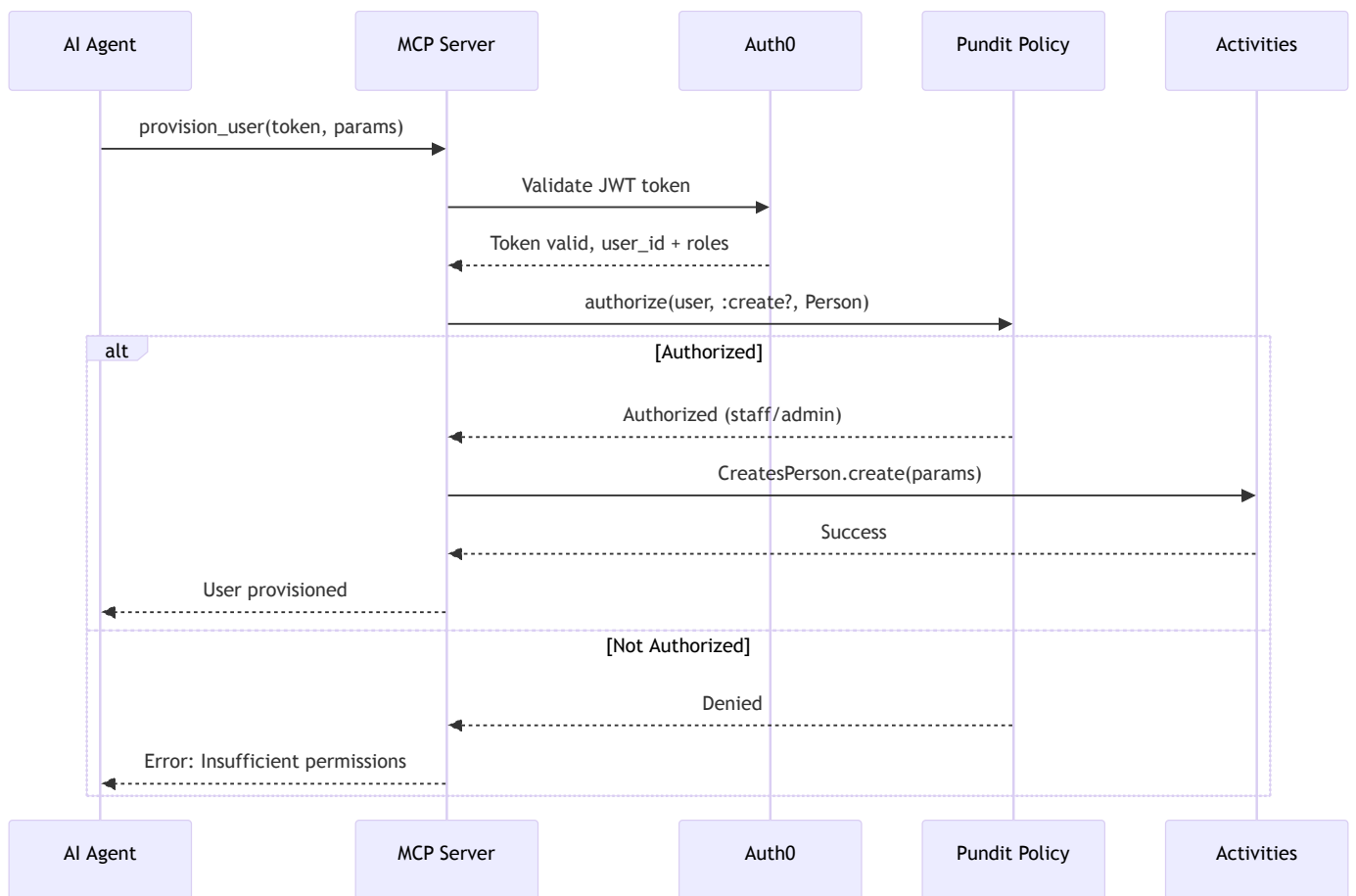
- Load testing (100 concurrent provisions)
- Failure scenario testing
- Rollback procedures
- Monitoring alert validation

Success Criteria:

-  MCP server deployed to production
-  All metrics flowing to Datadog
-  Error alerting configured
-  Operations team trained
-  AI agents provisioning live customers

Security & Compliance

Authentication & Authorization



Security Measures:

1. Authentication:

- Auth0 JWT Bearer tokens (existing mechanism)
- Token expiration enforced (1 hour default)
- Refresh token rotation
- API key authentication (for non-human agents)

2. **Authorization:**

- All tools enforce Pundit policies
- Role-based access control (RBAC)
- Resource-level permissions (account/location scoping)
- Staff-only tools (e.g., `activate_account` requires `staff` role)

3. **Audit Logging:**

- Every MCP tool invocation logged
- Logs include: timestamp, `user_id`, `tool_name`, parameters, result, duration
- Stored in database + Datadog
- Retention: 7 years (compliance requirement)

4. **Input Validation:**

- JSON Schema validation on all inputs
- Business rule validation in Activities
- SQL injection prevention (ActiveRecord parameterization)
- XSS prevention (input sanitization)

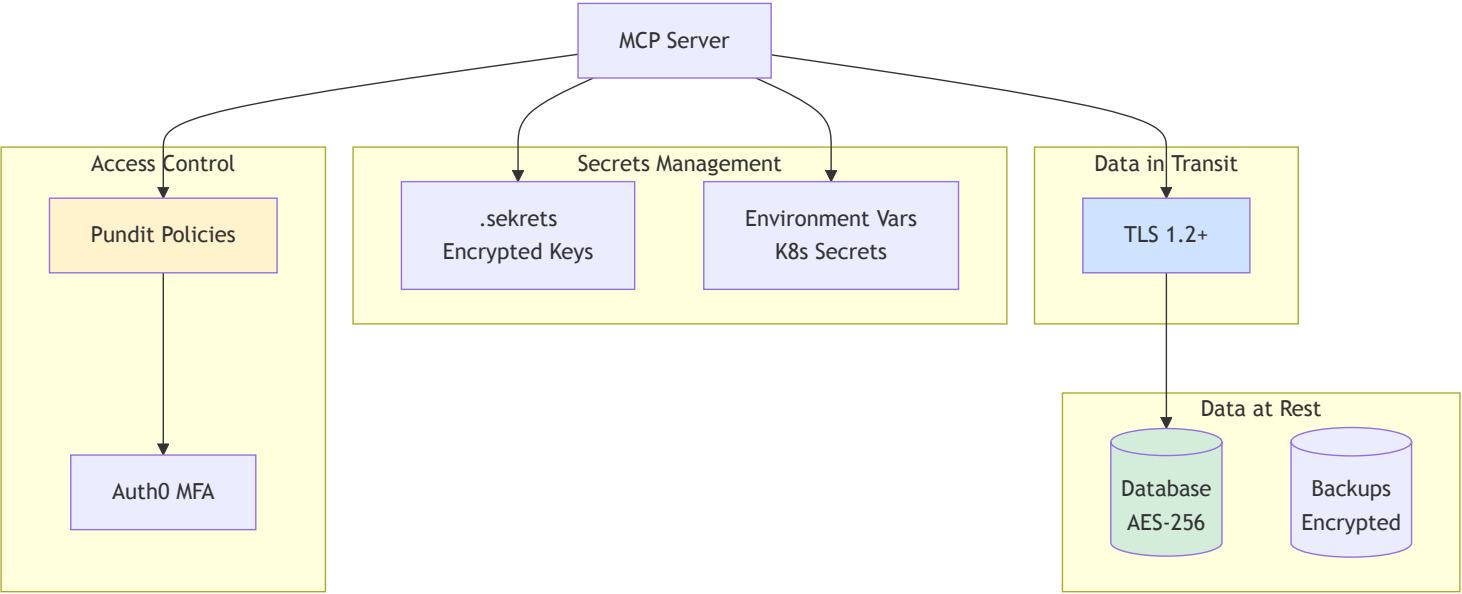
5. **Rate Limiting:**

- Per-user rate limits (100 requests/minute)
- Per-tool rate limits (`provision_account`: 10/hour)
- IP-based rate limiting for abuse prevention
- Exponential backoff on errors

Compliance Considerations

Regulation	Requirement	Implementation
SOC 2 Type II	Audit trail for all data changes	Comprehensive logging + database triggers
GDPR	Right to deletion, data portability	User deactivation tool, data export API
HIPAA (if applicable)	Encrypted PHI, access controls	TLS 1.2+, encryption at rest, RBAC
E911	Accurate address registration	Bandwidth E911 API integration
TCPA	Consent tracking for marketing	Stored in <code>person.consent_flags</code>

Data Protection



Risk Assessment

Risk Matrix

Risk	Probability	Impact	Severity	Mitigation
Data corruption from bypassed business logic	Low	Critical	HIGH	Use Activities layer exclusively, no direct DB writes
Auth0 sync failure	Medium	High	MEDIUM	Retry logic, dead letter queue, monitoring alerts
Rev.io billing sync failure	Medium	Critical	HIGH	Transactional rollback, manual reconciliation process
Kafka event emission failure	Low	High	MEDIUM	Local event store, replay mechanism
AI agent provides invalid	High	Low	LOW	JSON schema validation, clear

parameters				error messages
Concurrent provisioning race conditions	Medium	Medium	MEDIUM	Database transactions, optimistic locking
Production deployment downtime	Low	High	MEDIUM	Blue-green deployment, rollback plan
Unauthorized access	Low	Critical	HIGH	Auth0 JWT validation, Pundit policies, audit logging

Mitigation Strategies

1. Data Consistency Protection

Strategy: Wrap Activities in database transactions, implement rollback on external system failures

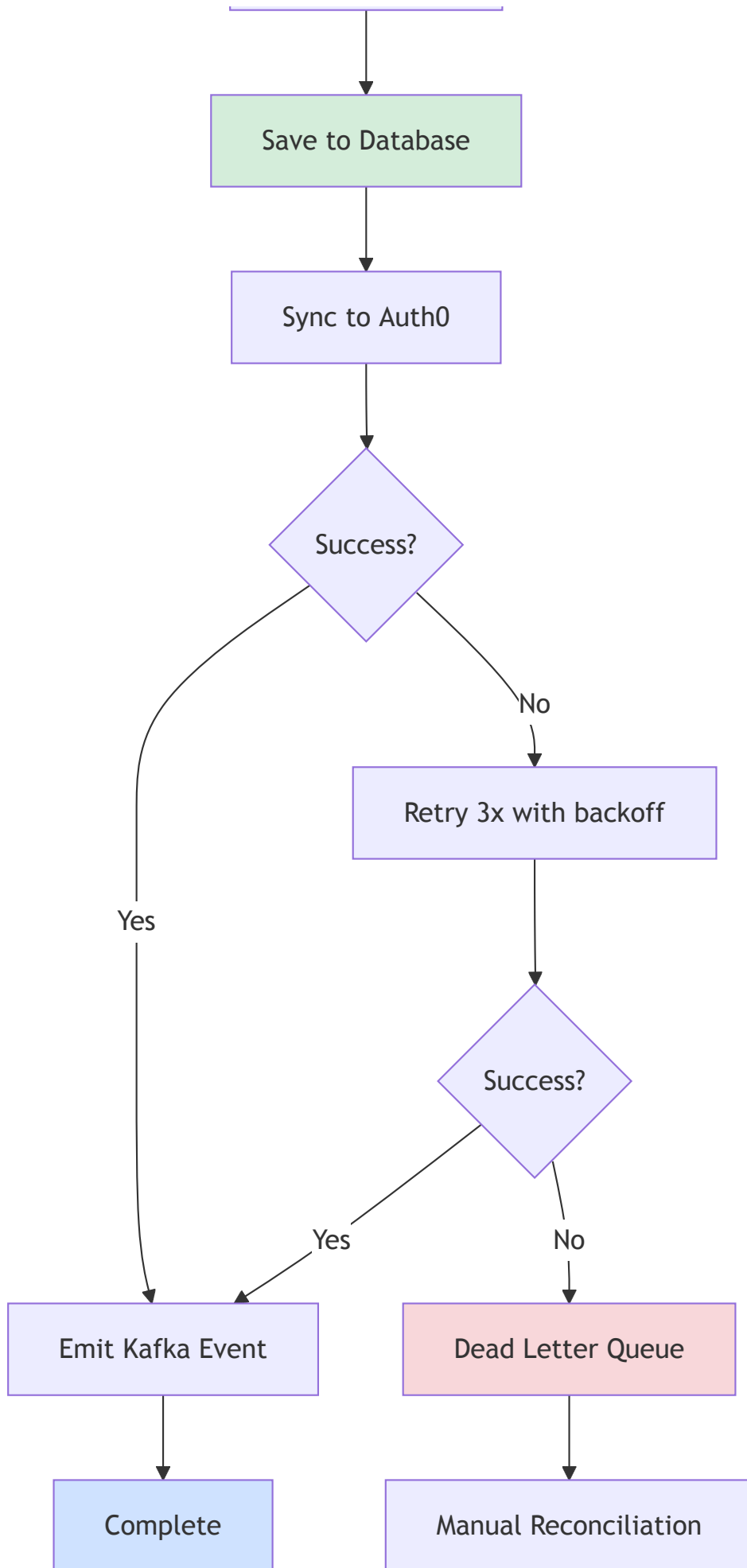
```
# Pseudocode
ApplicationRecord.transaction do
  account = Alps::Activities::CreatesAccount.create(params)

  begin
    sync_to_revio(account)
    sync_to_lms(account)
  rescue ExternalSystemError => e
    # Rollback database transaction
    raise ActiveRecord::Rollback
  end
end
```

2. External System Failure Handling

Strategy: Implement retry with exponential backoff, dead letter queue for permanent failures





3. AI Agent Error Recovery

Strategy: Provide actionable error messages with retry guidance

```
{
  "success": false,
  "error": {
    "code": "DUPLICATE_EMAIL",
    "message": "A user with email 'john@example.com' already exists",
    "field": "email",
    "suggestion": "Use a different email address or update the existing user",
    "retry": false,
    "existing_user_id": 12345
  }
}
```

4. Production Rollback Plan

Strategy: Blue-green deployment with instant rollback capability

Rollback Triggers:

- Error rate >5% for any tool
- Response time >30s (95th percentile)
- Auth0 sync failure rate >1%
- Database transaction failure rate >2%

Rollback Time: <5 minutes (DNS cutover)

Success Metrics

Key Performance Indicators (KPIs)

Operational Efficiency

Metric	Current (Manual)	Target (AI-Driven)	Measurement
Avg. Provisioning			Timestamp: quote signed →

Time	2-4 hours	<15 minutes	account active
Provisioning Error Rate	15%	<2%	% provisions requiring remediation
After-Hours Provisioning	0%	100%	% provisions completed 5pm-8am
Support Team Capacity	10 accounts/day	100 accounts/day	Max daily throughput

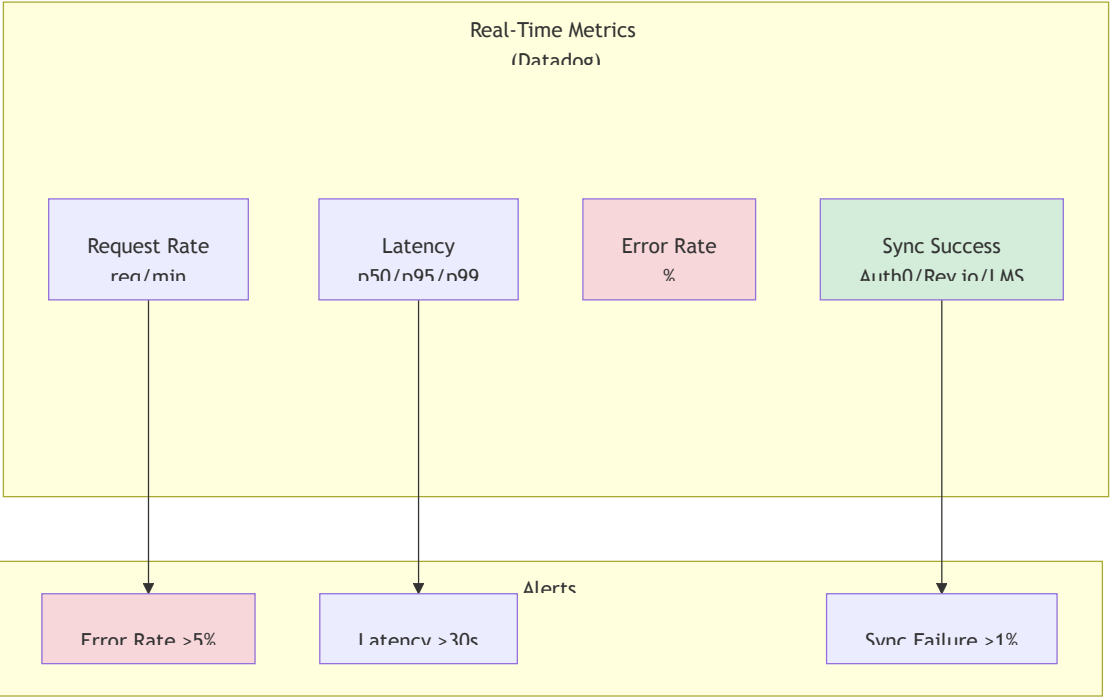
Quality Metrics

Metric	Target	Measurement
Data Consistency	100%	% provisions with all systems synced
Auth0 Sync Success	99.9%	% user provisions with Auth0 user created
Rev.io Sync Success	99.9%	% accounts with Rev.io customer record
Kafka Event Emission	100%	% provisions with events emitted
E911 Compliance	100%	% DIDs with valid E911 address

Business Impact

Metric	Current	Target (Year 1)	Measurement
Time-to-Revenue	2-4 hours	<1 hour	Quote signed → account active
Customer Satisfaction	N/A	>4.5/5	NPS score for onboarding
Operational Cost	>\$500K/year	\$50K/year	Labor + error remediation
Support Tickets	15% of provisions	<5%	% requiring manual intervention

Monitoring Dashboard



Weekly Reporting

Report Contents:

- 1. Total provisions (by AI agent vs. manual)
- 2. Success rate and error breakdown
- 3. Average provisioning time
- 4. External system sync rates
- 5. Cost savings realized
- 6. Customer feedback scores

Stakeholders: Engineering, Operations, Sales, Executive Team

Appendix: Technical Details

A. Database Schema (Key Tables)

```
-- accounts table (simplified)
CREATE TABLE accounts (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  domain VARCHAR(255) UNIQUE NOT NULL,
  account_number VARCHAR(50) UNIQUE,
```



```

account_type INT DEFAULT 1, -- 1=On-Boarding, 0=Active
billing_version INT DEFAULT 2,
customer_id VARCHAR(100), -- Rev.io customer ID
calling_pod_id INT,
time_zone VARCHAR(100) NOT NULL,
created_at TIMESTAMP,
updated_at TIMESTAMP,
INDEX idx_domain (domain),
INDEX idx_customer_id (customer_id)
);

-- people table (simplified)
CREATE TABLE people (
    id INT PRIMARY KEY AUTO_INCREMENT,
    account_id INT NOT NULL,
    location_id INT NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    login VARCHAR(255) UNIQUE NOT NULL,
    role VARCHAR(50) NOT NULL, -- user, manager, account_admin
    package VARCHAR(50) DEFAULT 'basic', -- basic, pro
    primary_line_id INT, -- SIP account
    vm_box_id INT,
    extension_id INT,
    code VARCHAR(50) UNIQUE, -- UUID
    created_at TIMESTAMP,
    updated_at TIMESTAMP,
    FOREIGN KEY (account_id) REFERENCES accounts(id),
    INDEX idx_email (email),
    INDEX idx_account (account_id)
);

-- subscriber table (SIP accounts, simplified)
CREATE TABLE subscriber (
    id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(100) NOT NULL,
    password VARCHAR(255) NOT NULL,
    domain VARCHAR(255) NOT NULL,
    address VARCHAR(255), -- sip:user@domain
    owned_by_type VARCHAR(50), -- Person, Location
    owned_by_id INT,
    nick VARCHAR(255), -- Display name
    caller_id_id INT,

```

```

vm_box_id INT,
is_webrtc BOOLEAN DEFAULT FALSE,
created_at TIMESTAMP,
updated_at TIMESTAMP,
UNIQUE KEY unique_username_domain (username, domain)
);

```

B. Activities Wrapper Pattern

```

# mcp/tools/user_management/provision_user.rb
module MCP
  module Tools
    module UserManagement
      class ProvisionUser < BaseTool
        def self.schema
          {
            name: 'provision_user',
            description: 'Create a new user with SIP line, extension, and voicemail'
            inputSchema: {
              type: 'object',
              properties: {
                account_id: { type: 'integer' },
                location_id: { type: 'integer' },
                first_name: { type: 'string', minLength: 1 },
                last_name: { type: 'string', minLength: 1 },
                email: { type: 'string', format: 'email' },
                role: { type: 'string', enum: ['user', 'manager', 'account_admin'] },
                package: { type: 'string', enum: ['basic', 'pro'] }
              },
              required: ['account_id', 'location_id', 'first_name', 'last_name', 'em
            }
          }
        end

        def execute(params, context)
          # 1. Authorization check
          authorize!(context.current_user, :create?, Person)

          # 2. Validate inputs
          validate_schema!(params)

          # 3. Fetch related resources
          account = Account.find(params[:account_id])

```

```

location = Location.find(params[:location_id])

# 4. Call Activities (proven business logic)
person = Alps::Activities::CreatesPerson.create(
  location,
  params.slice(:first_name, :last_name, :email, :role, :package),
  current_person: context.current_user
)

# 5. Return structured response
{
  success: true,
  person: {
    id: person.id,
    full_name: person.full_name,
    email: person.email,
    extension: person.extension&.number,
    sip_credentials: {
      username: person.primary_line.username,
      password: person.primary_line.password,
      domain: person.primary_line.domain,
      address: person.primary_line.address
    },
    voicemail_pin: person.personal_vm_box.pin
  }
}
rescue ActiveRecord::RecordInvalid => e
  # 6. User-friendly error handling
  {
    success: false,
    error: {
      code: 'VALIDATION_ERROR',
      message: e.message,
      fields: e.record.errors.messages,
      retry: false
    }
  }
end

private

def authorize!(user, action, resource_class)
  policy = Pundit.policy!(user, resource_class)
  raise Pundit::NotAuthorizedError unless policy.public_send(action)
end

```

```
    end
  end
end
end
end
```

C. Error Handling Strategy

Error Categories:

Category	HTTP Code	Retry	Example
Validation Error	400	No	Invalid email format
Authorization Error	403	No	Insufficient permissions
Resource Not Found	404	No	Account ID doesn't exist
Conflict	409	No	Duplicate email/username
External System Error	502	Yes	Auth0 API timeout
Transaction Error	500	Yes	Database deadlock
Rate Limit	429	Yes (backoff)	Too many requests

Error Response Format:

```
{
  "success": false,
  "error": {
    "code": "AUTH0_SYNC_FAILED",
    "message": "Failed to create Auth0 user after 3 retries",
    "details": "Connection timeout to auth0.com",
    "retry": true,
    "retry_after": 60,
    "suggestion": "Check Auth0 service status or retry in 1 minute",
    "support_ticket_id": "TKT-12345"
  },
  "data": null
}
```

D. Development Environment Setup

```
# 1. Install MCP Ruby gem (hypothetical – may need custom implementation)
gem install mcp-ruby

# 2. Create MCP server directory
mkdir -p mcp/provisioning_server
cd mcp/provisioning_server

# 3. Initialize MCP server
rails generate mcp:server provisioning

# 4. Start MCP server (development)
bundle exec ruby mcp/provisioning_server.rb

# 5. Test with MCP client
mcp-client connect --server stdio --command "ruby mcp/provisioning_server.rb"

# 6. Run tests
rspec spec/mcp/
```

E. Production Deployment (Docker/Kubernetes)

```
# k8s/mcp-provisioning-server.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mcp-provisioning-server
  namespace: bhive-production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mcp-provisioning
  template:
    metadata:
      labels:
        app: mcp-provisioning
    spec:
      containers:
        - name: mcp-server
          image: broadvoice/bhive-mcp-server:latest
```

```
ports:
- containerPort: 8080
env:
- name: RAILS_ENV
  value: production
- name: DATABASE_URL
  valueFrom:
    secretKeyRef:
      name: database-credentials
      key: url
- name: AUTH0_DOMAIN
  valueFrom:
    secretKeyRef:
      name: auth0-credentials
      key: domain
resources:
  requests:
    memory: "512Mi"
    cpu: "500m"
  limits:
    memory: "1Gi"
    cpu: "1000m"
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

```
apiVersion: v1
kind: Service
metadata:
  name: mcp-provisioning-service
  namespace: bhive-production
spec:
  selector:
    app: mcp-provisioning
  ports:
```

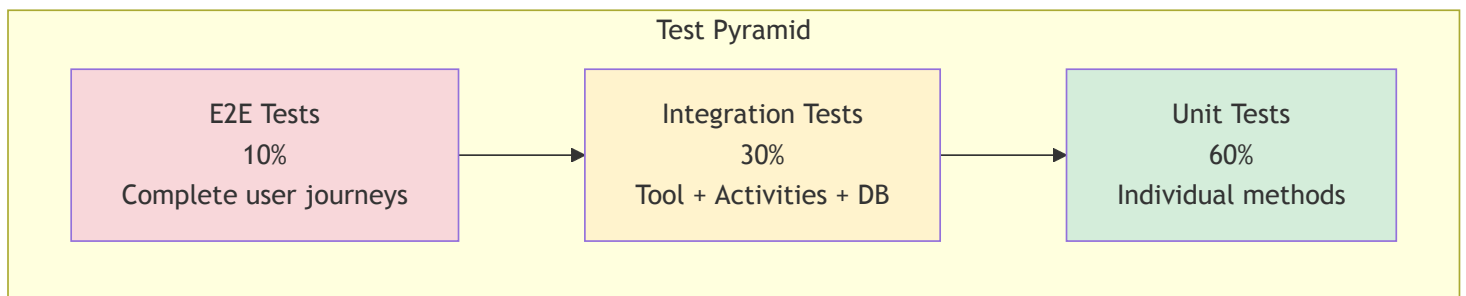
```
- protocol: TCP
  port: 80
  targetPort: 8080
type: LoadBalancer
```

F. Testing Strategy

Test Coverage Requirements:

- Unit tests: >90% coverage
- Integration tests: All tools with real database
- E2E tests: Complete provisioning flows
- Performance tests: <30s per provision (95th percentile)

Test Pyramid:



Example Test:

```
# spec/mcp/tools/user_management/provision_user_spec.rb
require 'rails_helper'
```

```
RSpec.describe MCP::Tools::UserManagement::ProvisionUser do
  let(:account) { create(:account, :active) }
  let(:location) { create(:location, account: account) }
  let(:admin_user) { create(:person, :account_admin, account: account) }
  let(:context) { double('Context', current_user: admin_user) }
```

```
  describe '#execute' do
    let(:params) do
      {
        account_id: account.id,
        location_id: location.id,
        first_name: 'John',
        last_name: 'Doe',
        email: 'john.doe@example.com',
```

```

    role: 'user',
    package: 'pro'
  }
end

it 'creates a user with SIP credentials' do
  result = described_class.new.execute(params, context)

  expect(result[:success]).to be true
  expect(result[:person][:email]).to eq('john.doe@example.com')
  expect(result[:person][:sip_credentials][:username]).to be_present
  expect(result[:person][:sip_credentials][:password]).to match(/\w{4}-\w{4}-\w{4}/)

  # Verify database records
  person = Person.find(result[:person][:id])
  expect(person.primary_line).to be_present
  expect(person.personal_vm_box).to be_present
  expect(person.extension).to be_present
end

it 'emits Kafka events' do
  expect(KafkaEventEmitter).to receive(:emit).with(
    topic: 'portal.alps.person-changed',
    subject: 'com.broadvoice.bhive.portal.alps.personChange',
    anything
  )

  described_class.new.execute(params, context)
end

it 'creates Auth0 user' do
  auth0_client = instance_double(Auth0::Client)
  allow(Auth0::Client).to receive(:new).and_return(auth0_client)
  expect(auth0_client).to receive(:create_user).with(
    email: 'john.doe@example.com',
    username: anything,
    password: anything
  ).and_return({ 'user_id' => 'auth0|12345' })

  described_class.new.execute(params, context)
end

context 'when email already exists' do
  before { create(:person, email: 'john.doe@example.com') }
end

```



```

it 'returns validation error' do
  result = described_class.new.execute(params, context)

  expect(result[:success]).to be false
  expect(result[:error][:code]).to eq('VALIDATION_ERROR')
  expect(result[:error][:fields][:email]).to include('has already been taken')
  expect(result[:error][:retry]).to be false
end

end

context 'when Auth0 sync fails' do
  before do
    allow_any_instance_of(Auth0::Client).to receive(:create_user).and_raise(Auth
  end

  it 'returns external system error' do
    result = described_class.new.execute(params, context)

    expect(result[:success]).to be false
    expect(result[:error][:code]).to eq('AUTH0_SYNC_FAILED')
    expect(result[:error][:retry]).to be true
  end

  it 'does not create person record (transaction rollback)' do
    expect {
      described_class.new.execute(params, context)
    }.not_to change(Person, :count)
  end
end

end

end
end

```

Conclusion

Summary

This plan proposes building an **MCP Server** to enable AI agent-driven customer provisioning for b-hive. By wrapping the existing, proven **Activities** business logic layer, we can:

1. **Preserve data consistency** across all systems (Auth0, [Rev.io](#), LMS, FreeSWITCH)
2. **Maintain security** through existing authorization mechanisms
3. **Reduce development time** (1-2 weeks vs 2-3 weeks for REST API)
4. **Provide AI-native interface** optimized for LLM interaction
5. **Enable 24/7 autonomous provisioning** without human intervention

Next Steps

1. **Executive Approval:** Review this plan with stakeholders
2. **Resource Allocation:** Assign 1 senior engineer for 4-5 weeks
3. **Phase 1 Kickoff:** Begin MCP server foundation and read-only tools
4. **Iterative Deployment:** Ship each phase to production incrementally
5. **Monitoring & Optimization:** Measure KPIs and iterate based on feedback

Expected Outcomes

- **80% reduction** in manual provisioning time
- **95% fewer** setup errors
- **10x increase** in onboarding capacity
- **\$130K annual savings** in operational costs
- **Improved customer satisfaction** through instant activation

Document Status: Draft for Review

Prepared By: João Camarate

Contact: jcamarate@broadvoice.com

Last Updated: October 14, 2025